

PM처럼 제품의 목적을 끝까지 파고들고, 디자이너처럼 작은 차이를 놓치지 않는 프론트엔드 개발자 권오승입니다.

기획서를 받으면 구현할 기능부터 확인하기보다, 사용자가 어떤 흐름을 거쳐 이 기능을 이용하게 될지 먼저 살펴봅니다. 사용 과정에서 불편할 만한 부분이 보이면 개선안을 제안하고, 디자이너와 대화하며 화면에 담긴 의도와 맥락까지 이해한 뒤 구현합니다.

프로젝트의 전 과정에 참여하다 보니 요구사항은 계속 달라질 수밖에 없다는 점도 배웠습니다. 기획이나 디자인이 바뀔 때마다 코드 곳곳을 수정하는 문제를 줄이기 위해 함수형 프로그래밍과 클린 아키텍처, FSD를 공부했습니다. 이를 반영한 구조를 팀에 제안해 실제 프로젝트에 적용했고, 현재는 전사 웹 프로젝트의 공통 개발 방식으로 사용하고 있습니다.

경험이 쌓이면서 제가 작성하는 코드뿐 아니라 팀이 반복해서 겪는 불편에도 관심을 갖게 됐습니다. 프로젝트마다 다시 만들던 UI를 디자인 시스템으로 정리하고, 화면 곳곳에 흩어져 있던 이벤트 로직은 선언형 트래킹 라이브러리로 통합했습니다. 최근에는 n8n과 AI 워크플로우를 활용해 반복 업무를 자동화하고 있습니다.

"저는 그동안 웹 주문과 결제, 상품권, 적립, 사장님 페이지, 디자인 시스템 등 회사의 주요 프로젝트를 맡아 진행했습니다. 현재는 Next.js 기반 서비스 개발을 중심으로 모니터링 시스템 구축과 배포 자동화, 웹 서버 운영까지 담당하며 제품의 개발부터 운영까지 함께하고 있습니다."

회사	경력	주요 영역
패스오더	Frontend Engineer · 4년	주문·결제 · 상품권 · 적립 · 백오피스 · 이벤트

패스오더 주요 프로젝트 12개 · 2023-2026

프로젝트 01

웹 주문·결제 시스템 구축

2026.07 - 2026.08

앱 설치 없이 메뉴 탐색부터 결제까지 완료하는 웹 주문 서비스 [실제 서비스 >](#)

성과 객단가 ×3 · 결제 전환 20% · 앱 설치 전환 32%

기술 React · TypeScript · FSD · Kakao Sync

서비스 기획 · 요구사항 정의 · 기술 설계 · 프론트엔드 개발

- BEFORE	웹에서 메뉴를 본 뒤 앱 설치 단계가 주문·결제 흐름을 단절
+ AFTER	웹 결제를 제공해 오가닉 유입에 대해 매출 전환을 시키고, 2차적으로 앱 설치를 통해 락인되는 구조 구축

프로젝트 기획

기존 웹은 메뉴 탐색까지만 가능했고, 주문하려면 앱을 설치해야 해 사용자 흐름이 끊기는 문제가 있었습니다. 이 문제를 해결하는 역할로 프로젝트에 투입됐고, 이후 서비스 기획과 일정 관리, 화면설계서 작성, 프론트엔드 설계·구현까지 전 과정을 담당했습니다.

앱 설치 없이 웹에서 주문과 결제를 완료할 수 있도록 전체 사용자 흐름과 기술 구조를 설계했습니다. 출시 후에는 **결제 전환율 20%**와 앱 대비 약 3배의 객단가를 기록했습니다.

주문·인증 흐름 설계

기존 웹에서는 매장과 메뉴만 볼 수 있었고, 주문하려면 앱을 설치해야 했습니다. 주문 단계가 늘어날수록 이탈 가능성이 커졌기 때문에 **인증 시점을 결제 직전으로 지연**시켰습니다. 외부 인증 페이지를 다녀와도 장바구니와 결제 입력값이 남도록 상태를 보존했습니다.

인증을 마치고 돌아오면 재고, 가격, 매장 상태, 회원 할인을 다시 확인해 품질되거나 금액이 바뀐 경우에는 변경 내용을 반영한 뒤 결제를 계속할 수 있게 했습니다.

프론트엔드 아키텍처 설계

결제 기능을 구현하면서 재고, 옵션, 가격, 할인처럼 서로 영향을 주는 정책과 예외 케이스가 빠르게 늘었습니다. 이 규칙을 화면이나 상태 관리 코드에서 직접 처리하면 정책이 바뀔 때 여러 컴포넌트를 함께 수정해야 했습니다.

FSD를 기준으로 도메인 규칙, 유즈케이스, UI의 책임을 나눴습니다. 재고 확인, 가격 계산, 할인 적용은 상태 관리와 분리된 순수 함수로 작성했습니다. 이후 정책 변경은 도메인 로직에서 처리하고 해당 함수만 테스트할 수 있었습니다. QA 중 정책이 바뀌었을 때도 UI 변경 범위를 줄였습니다.

출시 성과

출시 후 웹 주문 유저의 **객단가는 앱 유저의 약 3배**였습니다. 메뉴를 장바구니에 담은 사용자 가운데 **약 20%가 결제를 마쳤습니다.** 결제한 유저의 **32%가 앱을 설치**했고, 절반 이상은 마케팅 수신에 동의했습니다.

프로젝트 02

AI 개발 워크플로우와 품질 검증 자동화

2026.03 - 2026.05

요구사항 확인부터 코드 검증까지 AI 개발 과정을 표준화 [관련 기술 글](#)

성과 **AI 도구 실행 -34% · 작업 시간 -21% · Figma 전달 데이터 최대 -70%**

기술 **AI Workflow · Linter · Type Checker · PostToolUse Hook · Harness**

워크플로우 설계 · 개발 규칙 자동 검증 · Figma 데이터 후처리

- BEFORE **문서와 프롬프트에 의존해 개발 방식과 결과 품질이 작업자마다 달라짐**

+ AFTER **워크플로우와 하네스 엔지니어링을 구성해 결과물 품질을 높고 일관되게 개선함**

AI 개발 절차 표준화

사내에서 AI 도구를 쓰기 시작한 뒤 개발자마다 사용 방식이 달라, 같은 요구사항을 전달해도 개발 시간과 결과물 품질에 차이가 생겼습니다. 개발자가 여러 문서를 먼저 읽어야 했고, 문서 내용이 서로 다르거나 요구사항을 충분히 확인하지 않은 채 개발을 시작하는 경우도 있었습니다.

AI 워크플로우를 도입해 일관된 품질의 결과물을 만들도록 개선했습니다. AI에게 요구사항을 한 번에 넘기지 않고 단계별 필수로 확인해야 하는 문서들만 생성했습니다. 이로 인해 개발자가 작업 전에 읽어야 할 문서량을 기존의 **약 1/3**로 줄였습니다.

개발 규칙 자동 검증

워크플로우로 개발 절차를 통일시켰지만, AI 자체의 일관성 문제로 인해 프로젝트에서 쓰지 않는 패턴을 적용하거나 기존 규칙을 빠뜨리는 등의 문제가 발생했습니다. 하네스 엔지니어링을 적용해 문서와 프롬프트로 반복해서 설명하는 대신, 공통 개발 패턴을 Linter, Type Checker, Hook으로 검사했으며, 코드 레시피를 구성해 필수적으로 따라야 하는 지침을 구성했습니다.

이 개선을 통해 같은 작업에서 **AI 도구 실행 횟수는 34%, 작업 시간은 21%** 줄였습니다.

Figma MCP가 놓친 컴포넌트 정보 복원

Figma MCP를 연결해도 AI가 공통 컴포넌트 대신 새 UI를 만들거나 디자인과 다른 결과를 내놓았습니다. MCP가 전달한 데이터를 확인해 보니, 너무만 조금 달라져도 Figma 공통 컴포넌트로 인식되지 않았으며 반복되는 구조도 매번 전체 HTML로 전달되어 과도하게 토큰을 소모하고 있었습니다.

Figma API와 PostToolUse Hook으로 누락된 컴포넌트 정보를 복원하고 반복 데이터를 제거했습니다. 그 결과 Figma가 AI에 전달하는 입력 데이터가 **최대 70%** 줄었습니다.

프로젝트 03

친구 초대형 신규 사용자 획득 채널 구축

2026.03 - 2026.04

커피 쿠폰 공유부터 앱 설치·주문까지 이어지는 친구 초대 채널 구축

성과 **신규 가입자 목표 대비 16배 · 트래킹 공수 -60%**

기술 **React · Next.js · TypeScript · GA · AppsFlyer · Sentry**

이벤트 페이지 개발 · A/B 테스트 설계 · 선언형 트래킹 라이브러리 개발

- BEFORE	발신자·혜택이 불명확해 이벤트를 오해하고 트래킹 코드도 화면마다 반복
+ AFTER	혜택 중심 메시지로 개편하고 선언형 트래킹·A/B 테스트 환경 구축

기존 회원이 비회원인 친구에게 커피 쿠폰을 보내고, 수신자가 이벤트 참여부터 앱 설치와 주문까지 이어지는 신규 사용자 획득 채널을 개발했습니다. 메인 이벤트 페이지와 카카오톡 인중 플로우를 구현하고, 메시지와 화면 구성에 따른 전환 차이를 빠르게 확인할 수 있도록 A/B 테스트 구조를 함께 설계했습니다.

사용자 반응을 반영한 이벤트 개선

초기에는 발신자 정보 일부를 숨겨 수신자의 호기심을 유도했지만, 누구에게서 온 메시지인지 알기 어려워 이벤트의 의도를 오해하거나 불안함을 느끼는 사용자가 많았습니다.

사용자 반응과 전환 데이터를 바탕으로 방향을 다시 검토하고, **호기심보다 커피 쿠폰과 상품권 혜택을 명확하게 전달하는 방식으로 화면과 메시지를 개편**했습니다. 발신 정보와 혜택의 우선순위를 조정해 부정적인 반응을 줄였고, 이벤트 출시 이후 **신규 가입자 수 목표 대비 16배**를 달성했습니다.

실험·트래킹 환경 구축 및 전사 확장

A/B 테스트와 퍼널 분석을 위해 GA, AppsFlyer, 자체 행동 로그를 함께 사용하면서 트래킹 코드가 비즈니스 로직 곳곳에 반복적으로 들어가고, 이벤트 하나를 추가할 때마다 여러 코드를 수정해야 하는 문제가 생겼습니다.

이를 해결하기 위해 **프레임워크에 의존하지 않는 선언형 트래킹 라이브러리**를 단독 설계·개발했습니다. 마크업에 트래킹 정보를 선언하면 클릭·페이지
조화·콘텐츠 노출·스크롤 등의 이벤트를 감지하고, 어댑터를 통해 필요한 분석 도구로 전달하도록 구성했습니다.

이벤트 페이지에 먼저 적용한 뒤 다른 웹 서비스로 사용 범위가 확대됐고, **트래킹 코드 작성 공수를 약 60% 줄였습니다**. 또한 Sentry를 통해 오류와
주요 전환 흐름을 모니터링하며 트래픽이 집중되는 상황에서도 안정적으로 운영할 수 있도록 대응했습니다.

사용 기술: React, Next.js, TypeScript, GA, AppsFlyer, Web API, Adapter Pattern, Sentry

프로젝트 04

같이주문 서비스 · 결제 인프라 및 웹뷰 구축

2025.09 - 2025.12

사다리게임부터 주문·결제까지 이어지는 같이주문과 공통 웹 결제 기반 구축

성과 **사다리게임 결과 도달 목표 대비 130% · 홈 진입 152%**

기술 **WebView · Web Bridge · NICE · KICC · Token Auth**

웹 결제 기반 구축 · 공통 결제 패키지 설계 · 웹뷰 브릿지 구축

- BEFORE **PG·결제수단별 구현과 기능별 웹뷰 통신이 서비스 코드에 분산**

+ AFTER **PG 차이를 감춘 공통 결제 패키지와 이벤트 기반 웹↔앱 브릿지 구축**

여러 명이 주문을 공유하고 사다리게임으로 결제할 사람을 정한 뒤 실제 주문까지 이어지는 같이주문 서비스를 개발했습니다. **S2에서는 주문 공유·사다리
결과와 웹뷰 연동을, S3에서는 장바구니와 주문·결제를 담당**하며 기존 앱에서만 가능했던 결제를 웹으로 확장했습니다.

웹 결제 기반 및 공통 결제 패키지 구축

웹 결제를 처음 도입하면서 결제수단과 PG사마다 요청 방식과 결제창 동작이 달라, 각각의 차이를 서비스 화면에서 직접 처리하면 결제수단이
늘어날수록 구조가 복잡해지는 문제가 있었습니다.

다양한 결제수단과 PG사를 공통적으로 처리할 수 있는 결제 패키지를 설계해 복잡성을 줄였습니다. NICE·KICC 등 PG별 구현은 내부에서 분리하고
서비스에서는 동일한 방식으로 결제를 요청할 수 있도록 구성해, **같이주문뿐 아니라 이후 다른 웹 서비스에서도 재사용할 수 있는 공통 결제 시스템**을
만들었습니다.

또한 회원가입이 같이주문 참여의 진입 장벽이 되지 않도록 **비회원도 주문과 결제를 진행할 수 있는 토큰 기반 흐름**을 설계했습니다. 회원과 비회원의
인증·주문 정보를 구분하면서도 동일한 결제 흐름을 사용할 수 있도록 구성했습니다.

웹뷰 기반 같이주문 경험 구축

사다리게임과 주문 결과 화면을 웹뷰로 구현해 iOS와 Android에서 동일한 경험을 제공했습니다. 앱에서 선택한 멤버 정보를 웹으로 전달하거나 주문
상태 변화를 바로 반영해야 했기 때문에, 기능마다 별도의 통신 코드를 만드는 대신 **웹↔앱 브릿지 규격을 공통화하고 이벤트 기반으로 상태를 주고받는
구조**를 만들었습니다.

이를 통해 플랫폼별 중복 구현을 줄이고 이후 웹뷰 기능에서도 같은 통신 방식을 재사용할 수 있도록 했습니다. 서비스 도입 후 **사다리게임 결과 페이지
도달률은 설정 목표 대비 130%**를 기록했으며, **홈 진입 관련 지표도 152% 수준까지 증가**해 같이주문 이후의 재방문과 서비스 탐색을 유도했습니다.

웹 디자인 시스템 구축

2025.10 - 2025.11

여러 웹 서비스에서 재사용하는 컴포넌트와 Figma-AI 연계 기준 구축

성과 UI 개발 공수 10%+ 절감 · 2,000건 렌더 약 20배 개선

기술 Headless UI · Compound Component · Figma MCP · react-virtual

컴포넌트 정책 정의 · 공통 컴포넌트 설계·구현 · Figma-AI 워크플로우 연계

- BEFORE 서비스마다 UI를 중복 구현해 사용 방식과 디자인·코드가 불일치

+ AFTER Headless 공통 컴포넌트와 Figma·코드·AI 연결 기준 구축

사장님 웹, 플랫폼 웹, 백오피스에서 비슷한 UI를 각자 구현하면서 서비스마다 컴포넌트의 형태와 사용 방식이 달라지고, 디자인과 코드가 어긋나는 경우도 반복되고 있었습니다.

레거시 디자인 시스템의 개선 필요성을 제안했고 효율적인 부분을 인정받아 디자이너와 컴포넌트 정책, 공통 설계 원칙을 논의하고 Button, TextField, Dialog, Tabs, Table 등의 공통 컴포넌트를 설계·구현했습니다.

확장 가능한 컴포넌트 구조 설계

같은 컴포넌트라도 서비스마다 필요한 레이아웃과 기능이 달랐기 때문에, 요구사항이 추가될 때마다 props와 조건문을 늘리는 방식으로는 공통 컴포넌트를 유지하기 어렵다고 판단했습니다.

그래서 Headless 구조로 상태와 동작을 UI에서 분리하고, Compound Component 패턴으로 필요한 요소를 조합할 수 있도록 설계했습니다. HTML 기본 속성, className, forwardRef 등의 사용 방식도 공통 규칙으로 정리해 서비스별 확장은 허용하면서도 컴포넌트를 사용하는 방식은 일관되게 유지했습니다.

공통 컴포넌트의 변경은 별도 PR에서 리뷰하도록 하고, 기존 컴포넌트와 신규 디자인 시스템의 영역을 구분해 기존 서비스에 영향을 주지 않으면서 단계적으로 전환할 수 있는 운영 기준도 함께 정리했습니다.

Figma-AI 개발 워크플로우 연계

Figma와 코드에서 사용하는 컴포넌트명과 속성명이 달라, 디자인 구현 시 개발자가 적절한 컴포넌트를 다시 찾고 속성을 해석해야 했습니다. Figma MCP를 활용한 AI 개발에서도 같은 이유로 기존 공통 컴포넌트 대신 새로운 UI를 생성하는 문제가 있었습니다.

디자이너와 협업해 Figma 속성과 코드의 props·네이밍 기준을 맞추고, Figma 컴포넌트와 실제 코드 컴포넌트를 연결하는 기준을 정리했습니다. 이를 AI 개발 워크플로우와 연결해 디자인에 맞는 공통 컴포넌트를 더 정확하게 선택할 수 있도록 했고, UI 개발 공수를 10% 이상 줄였습니다.

대용량 테이블을 통한 확장성·성능 검증

디자인 시스템의 대표 컴포넌트로 백오피스용 대용량 테이블을 구현했습니다. 단순히 고정된 테이블 UI를 제공하기보다 Headless Hook과 Compound Component 구조를 결합해 톨바, 헤더, 그룹핑, 무한 스크롤 등을 화면에 따라 조합할 수 있도록 설계했습니다.

또한 react-virtual 기반 가상화와 메모이제이션을 적용해 화면에 필요한 행만 렌더링하도록 개선했습니다. 기존 구현과 비교한 결과 2,000건 이상의 실데이터 기준 초기 렌더 시간을 약 2.0초에서 0.1초 미만으로 단축해 약 20배 개선했습니다.

전자금융업 등록 대응 시스템 구축

2024.08 - 2025.02

규제 전용 백오피스와 공개 서비스·금융 시스템 분리 구조 구축

성과 전자금융업 등록 요건 충족 · 선불·정산 데이터 분리 운영 기반 마련

기술 iframe · postMessage · CSP

백오피스 프론트엔드 구축 · 금융 기능 보안 분리 · 민감정보 접근 통제

- BEFORE 공개 서비스에서 금융·민감 데이터를 함께 처리해 규제 요건 충족이 어려움

+ AFTER 전용 백오피스와 iframe-CSP 기반 금융 시스템 분리·접근 통제

전자금융거래법 개정에 따라 패스머니·상품권과 같은 선불수단과 PG 정산 데이터를 기존 서비스에서 분리해 관리해야 했습니다. 외부 통신이 제한된 환경과 공개 서비스·금융 시스템 간 분리 요건을 함께 충족하면서도, 운영자가 실제 정산 업무를 처리할 수 있는 별도 시스템이 필요했습니다.

전자금융업 백오피스를 초기 구축부터 서비스 전반까지 담당했고, 사장님 페이지에서 정산 관련 기능만 규정에 맞춰 안전하게 분리해 제공할 수 있도록 구조를 설계했습니다.

전자금융업 전용 백오피스 구축

선불수단의 충전·사용·잔액과 PG별 수수료·정산 내역을 조회하고, 지급대행 업무까지 처리할 수 있는 전용 백오피스를 구축했습니다. 금융 데이터를 다루는 화면이 빠르게 늘어나는 상황에서도 사용 방식이 달라지지 않도록 페이지 구성과 검색 조건 관리 방식을 공통화해 운영 화면의 일관성을 유지했습니다.

암/복호화를 적용해 민감정보를 기본 화면에서 마스킹하고, 원본 확인이 필요한 경우에만 파일로 내려받을 수 있도록 접근 경로를 분리했습니다. 다운로드 시 사용자와 사유를 함께 기록해 접근 이력을 남기고, 파일 자체에도 암호화를 적용해 조회 이후까지 민감정보 접근을 통제했습니다.

사장님 페이지 금융 기능 보안 분리

공개 웹서버인 기존 사장님 페이지가 금융 데이터를 처리하는 내부 시스템과 직접 통신할 수 없었기 때문에, 전자금융업 서버와 통신하는 별도 웹 페이지를 만들고 필요한 영역만 iframe으로 임베드해 보여주는 구조를 설계했습니다.

부모 페이지와 iframe은 postMessage를 통해 필요한 정보만 주고받도록 하고, 허용된 origin 검증과 CSP를 적용해 통신 범위를 제한했습니다. 인증이 완료된 이후에만 전자금융업 API를 호출하도록 구성해 공개 서비스와 금융 시스템 사이의 접근 경계를 명확히 했습니다.

또한 iframe 내부 높이를 동적으로 전달해 별도의 스크롤 영역 없이 기존 사장님 페이지 안에서 자연스럽게 동작하도록 구현했습니다. 이를 통해 기존 사용자 경험을 크게 바꾸지 않으면서도 금융 기능과 민감 데이터를 별도 시스템으로 분리할 수 있었습니다.

전자금융업 등록에 필요한 백오피스와 금융 기능 분리 구조를 구축해, 선불수단과 정산 데이터를 규제 요건에 맞춰 운영할 수 있는 기반을 마련했습니다.

B2C·B2B 상품권·선물하기 서비스 구축

2023.08 - 2025.02

상품권 생성·발행부터 B2B 대량 발송·정산까지 프론트엔드 구축

성과 패سو더의 사용 경험을 개인 주문 중심에서 선물 기반 경험으로 확장하고, 상품권 수신자를 신규 사용자로 연결

기술 React · Angular · Typescript · Dynamic Link · Deep Link

상품권 백오피스 구축 · B2C·B2B 수신 플로우 개발 · 정산 화면 구축

- BEFORE	패سو더 내 상품권 선물 기능이 없어 사용자 간 선물 경험과 신규 사용자 유입 경로가 제한적이었음
+ AFTER	상품권 구매·선물·수신·앱 연결까지 이어지는 선물하기 전체 플로우 구축

사용자가 상품권을 구매해 선물하고, 수신자가 웹에서 선물을 확인한 뒤 앱에 등록해 사용할 수 있는 상품권·선물하기 서비스를 개발했습니다. 운영팀이 상품권을 생성·발행하는 백오피스부터 B2C·B2B 수신자 웹과 앱 등록 플로우까지 상품권 관련 프론트엔드 전반을 담당했습니다.

서비스 출시 이후에도 상품권 유형 추가, 기업 대량 발송, 수신자 웹 개선, 정산 화면 구축 등 비즈니스 확장에 필요한 기능을 지속적으로 개발하고 운영했습니다.

상품권 발행·수신 플로우 구축

운영팀이 상품권을 생성·수정하고 발행할 수 있는 백오피스와, 수신자가 웹에서 선물받은 상품권을 확인하고 앱에 등록하는 전체 플로우를 구축했습니다.

B2B 대량 발송 구조 확장

기존 B2C 선물하기는 결제와 함께 주문이 생성되기 때문에 발급된 상품권 식별자를 기준으로 수신 정보를 조회할 수 있었습니다. 반면 기업이 여러 명에게 상품권을 발송하는 B2B 방식은 개별 주문 없이 수신자의 이름과 연락처를 기준으로 발행하는 구조라 기존 방식을 그대로 사용할 수 없었습니다.

이에 **B2C와 B2B의 수신 플로우를 분리**했습니다. 주문 정보가 없는 경우에도 수신자를 안전하게 식별하고, 웹에서 상품권을 확인한 뒤 앱 등록까지 이어질 수 있도록 구성했습니다.

백오피스에는 수신자 목록의 유효성을 검증하고 여러 사용자에게 상품권을 일괄 발송할 수 있는 기능을 추가해 기업 대상 상품권 운영까지 지원했습니다.

운영·정산 영역 확장

서비스가 확장되면서 시존 상품권과 B2B 발행 유형, 판매 상태와 발송 내역 관리 기능을 지속적으로 추가했습니다. 이용안내와 약관은 클라이언트 코드에 직접 포함하던 방식에서 서버에서 관리하는 구조로 전환해, 정책이 변경될 때마다 프론트엔드를 다시 배포해야 하는 의존성을 줄였습니다.

전자금융업 시스템 분리 이후에는 **상품권 구매와 프로모션 데이터를 월별로 확인할 수 있는 정산·재무제표 화면**도 구축했습니다. 일반적인 목록형 데이터와 다른 정산 구조를 별도의 변환 로직으로 분리해, 항목이 추가되더라도 화면 전체를 수정하지 않고 대응할 수 있도록 했습니다.

이를 통해 상품권의 생성·발행부터 수신·앱 등록, 기업 대량 발송과 정산까지 이어지는 **B2C·B2B 선물 커머스 운영 기반**을 구축했습니다.

사용 기술: React, TypeScript, Angular, Dynamic Link, Deep Link, Web API

미가맹 매장 획득을 위한 사전등록 시스템 구축

2023.04 - 2024.07

미가맹 매장 노출과 사용자 참여 데이터를 후속 영업으로 연결

성과 **매장 도입률 5%+**기술 **React · TypeScript · Firebase Remote Config · WebView**

사용자 참여 기능 개발 · 웹뷰 외부 서비스 연동 · 운영 백오피스 구축

- BEFORE 공공데이터 기반 미가맹 매장 노출만으로는 사용자 반응을 영업에 활용하기 어려움**+ AFTER** 전화 주문·리뷰 참여 데이터와 운영 백오피스를 후속 영업 흐름에 연결

공공데이터와 외부 데이터를 기반으로 미가맹 매장을 앱에 먼저 노출하고, 이를 운영·관리할 수 있는 백오피스와 사용자 참여 기능을 개발했습니다. 사용자의 전화 주문과 영수증 리뷰를 통해 미가맹 사장님에게 패스오더를 알리고, 축적된 참여 데이터를 영업 조직의 후속 접근으로 연결하는 매장 획득 프로젝트입니다.

미가맹 매장 노출 및 사용자 참여 기능 개발

사용자가 주변의 미가맹 매장을 쉽게 발견하고 참여할 수 있도록 매장 카드와 상세 화면, 주변 매장 탐색, 매장 정보 제보 기능을 구현했습니다. 사전등록 매장과 일반 입점 매장을 구분할 수 있도록 상태와 배지 정보를 제공하고, 다양한 경로에서 미가맹 매장을 발견할 수 있도록 노출 영역을 확장했습니다.

전화 주문 이후에는 사용자가 영수증 리뷰까지 자연스럽게 이어갈 수 있도록 인앱 웹뷰에서 외부 리뷰 서비스를 연동했습니다. 외부 페이지 변경에도 앱 재배포 없이 대응할 수 있도록 **Firebase Remote Config** 기반으로 연동 로직을 관리해 문제가 발생하면 원격에서 수정하거나 비활성화할 수 있도록 구성했습니다.

매장 획득 데이터·운영 백오피스 구축

사전등록 매장의 상태와 사용자 반응을 운영자가 한눈에 확인할 수 있도록 전용 백오피스를 구축했습니다. 매장별 전화 시도와 성공 건수 등 **사용자 참여 성과를 조회할 수 있는 화면**을 만들고, 운영 기간과 상태, 리뷰드 등급에 따라 매장을 필터링할 수 있도록 구성했습니다.

반복되는 매장 정보와 운영 상태는 카드형 UI로 정리하고, 목록과 로딩 영역은 공통 컴포넌트로 분리해 기능이 추가돼도 동일한 구조를 재사용할 수 있도록 했습니다.

이를 통해 파지팁과 영업 조직이 단순히 미가맹 매장 목록을 확인하는 데 그치지 않고, **사용자 반응이 높은 매장을 우선적으로 선별해 후속 영업에 활용할 수 있는 운영 구조**를 만들었습니다.

성과

미가맹 매장의 사전 노출부터 사용자 참여, 운영 데이터 확인, 후속 영업까지 이어지는 흐름을 구축해 **매장 도입률 5% 이상**을 달성했습니다.

공공데이터로 확보한 매장 정보를 단순 노출하는 데 그치지 않고, **사용자 참여 → 사장님의 서비스 인지 → 영업 조직의 후속 접근**으로 이어지는 신규 매장 획득 채널로 확장했습니다.

사용 기술: React, TypeScript, Firebase Remote Config, WebView

패스링크·자체 딥링크 플랫폼 구축

2024.03 - 2024.05

외부 딥링크를 대체해 링크 생성·변경·성가를 내부 운영

성과 외부 딥링크 의존도 축소·링크 변경과 유입 성과 확인을 내부 운영

기술 React · TypeScript · Nginx · Universal Links · App Links · Web API

딥링크 웹 구축·링크 관리 백오피스 개발·클릭 성과 측정

- BEFORE MAU 과금 외부 딥링크에 의존하고 목적지 변경마다 링크·메시지를 재검수

+ AFTER 자체 링크 래핑·환경별 fallback·성과 관리 백오피스 구축

CRM 메시지와 앱 푸시, 알림톡·친구톡에서 사용하던 Branch와 Firebase Dynamic Links 등 외부 딥링크 서비스는 실제 링크 사용량이 아닌 전체 앱 MAU를 기준으로 비용이 발생했습니다. 또한 목적지나 외부 플랫폼이 변경될 때마다 여러 채널에 흩어진 링크를 찾아 수정하고 메시지 템플릿을 다시 검수해야 했습니다.

이러한 운영 비용과 외부 서비스 의존도를 줄이기 위해 자체 딥링크 서비스인 '패스링크'의 웹 페이지와 관리 백오피스를 구축했습니다.

외부 딥링크를 대체하는 링크 래핑 구조 구축

CRM과 메시지 템플릿에는 실제 목적지 대신 패스링크 주소를 사용하고, 내부에서 앱·웹 목적지를 관리하는 래핑 구조를 적용했습니다.

목적지나 딥링크 플랫폼이 변경되더라도 이미 발송되거나 템플릿에 등록된 링크를 다시 교체하지 않고, 패스링크의 내부 매핑만 수정해 새로운 주소로 연결할 수 있도록 설계했습니다. 이를 통해 외부 서비스에 분산돼 있던 링크를 하나의 시스템으로 통합하고, 플랫폼 변경 시 발생하던 링크 전수 교체와 메시지 템플릿 재검수 부담을 줄였습니다.

OS·앱 설치 상태별 연결 처리

하나의 링크로 접속하더라도 사용자의 OS와 앱 설치 여부, 브라우저 환경에 따라 이동해야 하는 목적지가 달랐습니다.

iOS의 Universal Link와 Android의 App Link를 기반으로 앱 실행을 처리하고, 앱이 설치되지 않은 경우에는 스토어나 웹 페이지로 이동하도록 fallback 흐름을 구성했습니다. 또한 모바일·데스크톱과 인앱 브라우저 등 환경별 차이를 고려해 사용자가 어떤 경로에서 진입하더라도 적절한 목적지로 연결될 수 있도록 예외 상황을 처리했습니다.

앱과 도메인의 연결 설정은 Nginx에서 관리해 외부 딥링크 서비스에 의존하지 않고 자체 도메인 기반으로 앱 연결을 운영할 수 있도록 했습니다.

링크 운영·성과 측정 백오피스 구축

운영자가 개발자의 도움 없이 링크를 생성하고 목적지를 변경할 수 있도록 관리 백오피스를 구축했습니다. 링크의 사용 여부와 운영 기간, 앱·웹 fallback 주소를 직접 설정하고, CRM에서 반복적으로 사용하는 링크는 템플릿 단위로 관리할 수 있도록 구성했습니다.

또한 템플릿과 개별 링크 단위로 클릭 데이터를 집계해 어떤 메시지와 캠페인이 실제 사용자 유입으로 이어졌는지 확인할 수 있도록 했습니다. 축적된 데이터는 이후 그로스팀이 CRM 성과를 분석하고, 설치 이후 사용자 이동까지 추적하는 기능으로 확장할 수 있는 기반이 됐습니다.

이를 통해 MAU 규모에 따라 비용이 증가하던 외부 딥링크 서비스의 의존도를 낮추고, 링크 생성과 변경부터 유입 성과 확인까지 내부에서 직접 운영할 수 있는 구조를 마련했습니다.

사용 기술: React, TypeScript, Nginx, Universal Links, Android App Links, Web API

카카오싱크 기반 비회원 이벤트 참여 플로우 개선

2024.02 - 2024.04

카카오싱크와 퍼머링크로 신규 사용자 인증·복귀 흐름 개선

성과 친구 초대 성공률 32% → 38% (+6%p)

기술 React · Kakao Sync · WebView

인증 플로우 설계 · 웹뷰 분기 처리 · iOS 복귀 병목 개선

- BEFORE 휴대폰 번호 인증과 iOS 수동 복귀로 참여 단계가 늘고 복귀 구간에서 이탈

+ AFTER 카카오싱크 인증과 퍼머링크 자동 복귀로 참여 단계 단축

카카오싱크 기반 이벤트 참여 인증 간소화

기존 이벤트는 사용자가 휴대폰 번호를 직접 입력하고 인증해야 참여할 수 있어, 신규 유입 사용자가 이벤트에 진입하기까지 여러 단계를 거쳐야 했습니다.

이벤트 참여 인증 플로우를 카카오싱크 기반으로 전환해, **카카오 인증만으로도 이벤트에 필요한 사용자 정보 수집이 가능하도록** 구성했습니다. 웹앱/이벤트 서비스에서 회원 여부와 유입 경로에 따른 인증 분기를 처리해, 기존 휴대폰 번호 인증보다 참여 과정을 간소화하고 전반적인 이벤트 진입 장벽을 낮췄습니다.

퍼머링크 도입을 통한 iOS 복귀 병목 개선

iOS에서는 카카오톡 인증을 마친 뒤 브라우저로 자동 복귀하지 않아, 인증을 완료한 사용자도 이벤트 참여를 이어가지 못하고 이탈하는 문제가 있었습니다.

사용자의 실제 이동 과정을 분석해 인증 후 복귀 구간을 병목으로 특정하고, **카카오싱크와 연계된 퍼머링크 복귀 방식을 제안·적용**했습니다. 인증이 끝나는 즉시 사용자를 이벤트 결과 화면으로 돌려보내도록 플로우를 변경해, 수동으로 브라우저를 찾아 돌아와야 했던 불편을 해소했습니다.

성과

퍼머링크 적용 이후 인증부터 결과 화면까지의 흐름이 끊기지 않으면서 **친구 초대 이벤트 참여 성공률이 32%에서 38%로 6%p 상승**했습니다. 또한 휴대폰 번호를 직접 입력하던 참여 방식을 카카오싱크 인증으로 간소화해, 신규 사용자의 이벤트 참여 허들을 낮추고 전반적인 참여율을 개선했습니다.

사용 기술: React, Kakao Sync, WebView

사장님 가입·계약 신청서 구축 및 개선

2023.09 - 2024.01

사용자 유형별 계약 플로우와 오류 대응 UX를 갖춘 모바일 다단계 신청서 구축

성과 개선 버전 배포 당일 접수 이슈 없이 안정적으로 출시

기술 React · TypeScript · Multi-step Form · Web API

- BEFORE	고정된 신청 절차와 모호한 오류 안내로 작성 중단·현업 문의 발생
+ AFTER	사용자 유형별 단계·이어쓰기와 필드 단위 오류 안내·복구 흐름 구축

패스워드 도입을 희망하는 사장님이 매장과 사업자, 정산·결제 정보를 입력하고 가입과 계약을 완료하는 모바일 신청서를 개발했습니다. 초기 신청서와 영업용 플로우의 프론트엔드 개발을 담당했고, 이후 개선 스프린트에서는 기존 구조를 재정비하며 개발부터 QA·배포까지 수행했습니다.

배포 이후로도 파트너지원팀과 영업팀과 지속적으로 소통하며 지속적으로 신청서를 개선해 신청 절차를 간소화 하고 입력 오류 정보를 명시적으로 제공해 신청서 작성 인입/중단율을 낮췄습니다.

사용자 유형별 계약 플로우 설계

일반 신청서와 가오픈·영업용 신청서는 매장 생성 시점과 기존에 저장된 정보가 달라 하나의 고정된 가입 절차로 처리하기 어려웠습니다.

사용자 유형과 신청 상태에 따라 필요한 단계만 노출하고, 각 단계의 입력값을 저장해 중단된 지점부터 이어서 작성할 수 있는 다단계 플로우를 설계했습니다. 기존 계정이나 신청 이력이 있는 경우에도 단순히 차단하지 않고 로그인이나 기존 신청서 이어쓰기로 연결했습니다.

또한 실제 사업자 서류 기준과 입력 항목을 맞춰 접수 과정에서 발생하던 정보 불일치를 줄였습니다.

입력·인증 오류 UX 개선

기존 신청서는 입력 오류가 발생해도 어떤 항목이 문제인지 알기 어려워 현업 문의로 이어지는 경우가 많았습니다.

필수·선택 항목을 명확히 구분하고, 유효성 검사에 실패하면 오류 원인을 안내한 뒤 해당 입력 필드로 자동 이동하도록 개선했습니다. 휴대폰 인증도 발송·입력·만료·불일치·재전송 상태를 구분하고, 상황에 맞는 안내 문구와 타이머를 제공했습니다.

API 오류 역시 상황별 메시지로 구분해 사용자가 직접 해결할 수 있도록 하고, 영업 담당자도 전달받은 정보만으로 원인을 파악할 수 있게 했습니다.

현업 피드백 기반 리팩터링과 배포

초기 일정에 맞춰 빠르게 구현된 코드에 사용자 유형과 예외 케이스가 계속 추가되면서 상태 분기와 검증 로직이 복잡해졌습니다.

개선 스프린트에서는 단계별 폼 상태, 저장 로직, 유효성 검사, API 오류 처리를 역할별로 분리해 구조를 다시 정리했습니다. 파트너지원팀의 문의 내용과 영업팀의 실제 사용 경험을 바탕으로 개선 항목을 정리하고, 사용자 유형별 가입·이어쓰기·인증 플로우를 중심으로 QA를 진행했습니다.

배포 시에는 변경 사항과 오류 대응 방법을 담은 릴리즈 노트를 직접 작성해 현업 부서에 공유했고, 개선 버전 배포 당일 접수 이슈 없이 안정적으로 출시했습니다.

사용 기술: React, TypeScript, Multi-step Form, Web API

같이적립 · 포인트 공유 기반 신규 사용자 획득 채널 구축

2023.01 - 2023.03

매장 포인트를 친구에게 선물하는 같이적립 웹 전반과 사용자 상태별 수신 플로우 구축

성과 **포인트 수령에서 앱 유입·가입까지 이어지는 주요 획득 채널로 활용**기술 **React · TypeScript · Kakao Sync · Deep Link · GA · Web API**

사용자 상태별 수신·가입 플로우 설계 · 공유 기능 개발 · 채팅형 수신 UI 구현

- BEFORE **연락처 동기화만으로 포인트를 보내고 사용자 상태마다 수신 경로가 달라짐****+ AFTER** **링크 공유와 상태별 인증·앱 이동·적립 수신 플로우 구축**

사용자가 매장에서 적립한 포인트를 친구에게 선물하고, 수신자가 웹에서 포인트를 확인한 뒤 앱 설치와 가입까지 이어지는 같이적립 서비스를 개발했습니다. 같이적립 관련 웹 페이지 전반을 담당하며, 사용자 상태와 진입 경로에 따라 달라지는 수신·가입 흐름을 설계했습니다.

또한 기존 연락처 동기화 방식 외에 **링크만으로도 포인트를 선물할 수 있는 공유 기능**을 추가해 주소록 접근에 대한 부담 없이 더 많은 사용자에게 서비스를 확산할 수 있도록 했습니다.

사용자 상태별 적립 수신·가입 플로우 설계

회원 여부와 앱 설치 상태, 공유 링크 진입 여부에 따라 필요한 행동이 달랐습니다. 회원은 앱에서 바로 적립을 확인하도록 연결하고, 비회원은 카카오톡 인증을 거쳐 웹에서 적립 내역을 확인한 뒤 앱 설치와 가입으로 이어질 수 있도록 구성했습니다.

공유 링크로 진입한 경우에도 앱 설치 여부를 확인해 **설치된 사용자는 앱의 해당 화면으로, 미설치 사용자는 웹 인증과 적립 확인으로 연결**했습니다. 이를 통해 회원·비회원·앱 미설치 사용자 모두 자신의 상태에 맞는 경로로 적립을 받을 수 있도록 했습니다.

사용자 상태에 따른 분기가 화면 코드에 뒤섞이지 않도록 **인증·앱 이동·적립 수신 로직의 역할**을 분리하고 **공통 플로우를 재사용**해, 이후 인증 정책이나 화면 변경에도 필요한 영역만 수정할 수 있도록 구성했습니다.

공유·수신 경험 개선

포인트를 선물받은 사용자가 왜 이 페이지에 들어왔고 다음에 무엇을 해야 하는지 자연스럽게 이해할 수 있도록 **발신자와 수신자의 상황을 대화 형식으로 보여주는 채팅형 UI**를 구현했습니다.

회원 여부와 적립 상태에 따라 대화 내용과 CTA를 다르게 노출해 각 사용자에게 필요한 행동만 보여줬고, 링크 공유 기능을 함께 제공해 연락처 동기화를 원하지 않는 사용자도 별도의 설정 없이 여러 친구에게 포인트를 보낼 수 있도록 했습니다.

출시 이후 같이적립은 **포인트 수령에서 앱 유입·가입까지 이어지는 주요 획득 채널로 활용**됐으며, **신규 사용자 확보에 유의미하게 기여**했습니다. 회원·비회원·공유 링크별 예외 상황을 사전에 검증하고 개발자 QA를 진행해 첫 배포 이후 관련 CS도 거의 발생하지 않았습니다.

사용 기술: React, TypeScript, Kakao Sync, Deep Link, GA, Web API